

RWTH CLAIX cheat sheet

Login, file transfer and the Slurm basics for running the 1D-PDPTW solver. Account: **fhy0180**. Replace <...> placeholders. Commands marked [verify] depend on site setup: check the ITC docs (help.itc.rwth-aachen.de) if they fail.

1. Logging in (SSH)

You must be in the RWTH network or connected to the **RWTH VPN** (needed from home / eduroam-less networks).

```
ssh fhy0180@login23-1.hpc.itc.rwth-aachen.de    # log in
exit                                           # log out (or Ctrl+D)
```

Optional: passwordless login with an SSH key (Windows PowerShell / Linux / macOS):

```
ssh-keygen -t ed25519                        # create key (once), accept defaults
ssh-copy-id fhy0180@login23-1.hpc.itc.rwth-aachen.de # Linux/macOS
# PowerShell has no ssh-copy-id: append your public key manually:
$k = Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub
# (uses the 'claix' alias from ~/.ssh/config below, or type the full host)
ssh claix "echo '$k' >> ~/.ssh/authorized_keys"
```

Shortcut: put this in ~/.ssh/config (on your PC), then just type `ssh claix`:

```
Host claix
  HostName login23-1.hpc.itc.rwth-aachen.de
  User fhy0180
```

Do not run heavy computations on the login node: use `sbatch` (section 4).

2. Upload: local PC → CLAIX

Run these on your **local** machine (PowerShell/terminal), not inside the SSH session.

```
# one file
scp instance.json fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw/instances/

# a whole folder (recursive)
scp -r .\pdptw_project fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/

# many files / repeated sync, only sends changes (Linux/macOS/WSL/Git-Bash)
rsync -avP ./pdptw_project/ fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/
```

Large files / many instances: use the dedicated data-transfer node if the login node is slow [verify] and keep big data in your project/work directory rather than \$HOME (check quota with `r_quota` or the docs). At the chair, very large files usually go through sciebo (see ITC sciebo page).

3. Download: CLAIX → local PC

```
# one file
scp fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/results/out_xyz.json .

# whole results folder
scp -r fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/results ./results_claix

# only new/changed files
rsync -avP fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/results/ ./results_claix/

# logs of a job
scp "fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/logs/*.out" ./logs/
```

Tip: quote paths containing % or spaces, e.g. `"bar-n100-1_node_restriction_percentage_50%_loop.json"`.

4. Slurm: running jobs

```
mkdir -p logs
sbatch run_extended_v4.sh          # submit (prints: Submitted batch job 1234567)
sbatch --array=0 run_extended_v4.sh # only task 0 of the array
squeue -u $USER                    # my jobs (PD = pending, R = running)
scontrol show job <jobid>          # details, incl. pending reason
scancel <jobid>                    # cancel one job
scancel -u $USER                   # cancel all my jobs
sacct -j <jobid> --format=JobID,State,Elapsed,MaxRSS,ExitCode # after it finished
tail -f logs/1dpdptw_ext_<jobid>_0.out # watch the live log (Ctrl+C to stop watching)
```

Key #SBATCH options:

Option	Meaning
--account=<project>	CLAIX project to bill (required)
--time=HH:MM:SS	wall-clock limit; job is killed after this
--cpus-per-task=8	cores; must match Gurobi Threads (8)
--mem=64G	RAM for the job (NodefileStart=20 GB in v4)
--array=0-N	N+1 independent tasks; index in \$SLURM_ARRAY_TASK_ID
--signal=TERM@300	SIGTERM 300 s before the limit (v4 exports the incumbent)
--output / --error	log files; %x = name, %A = job id, %a = task id

Interactive test (short, for debugging only) **[verify partition/limits]**:

```
salloc --account=<project> --cpus-per-task=4 --mem=16G --time=00:30:00
srun --pty bash # shell on a compute node; 'exit' to release
```

5. Environment (modules, Python, Gurobi)

```
module avail # list modules
module spider Python # search incl. required toolchains
module spider Gurobi # is there a Gurobi module on CLAIX?
module load <name/version> # load
module list # what is loaded
module purge # unload everything

# one-time Python venv
module load GCCcore Python # use exact versions from 'module spider'
python -m venv ~/venvs/pdptw
source ~/venvs/pdptw/bin/activate
pip install gurobipy # licence: academic/WLS licence needed on the cluster [verify]
python -c "import gurobipy; print(gurobipy.gurobi.version())"
```

6. Basic Linux commands

Command	What it does
pwd / ls -lah / cd <dir> / cd ..	where am I / list files / change directory / up
mkdir -p a/b / rm file / rm -r dir	create dir / delete file / delete dir (no undo!)
cp a b / cp -r d1 d2 / mv a b	copy / copy dir / move or rename
cat f / head -n 20 f / tail -n 50 f	print / first / last lines
tail -f f	follow a growing log file
grep -n 'text' f / grep -r 'x' dir	search in file / recursively
nano f (Ctrl+O save, Ctrl+X quit)	simple editor; vim also available
du -sh * / df -h / r_quota	folder sizes / disk free / quota [verify r_quota]
htop / top (q to quit)	processes and memory

<code>chmod +x script.sh</code>	make a script executable
<code>tmux / tmux attach</code>	session that survives disconnects (login node only for light work)
<code>unzip a.zip / tar xzf a.tar.gz</code>	extract archives
<code>Ctrl+C / Ctrl+D / clear</code>	interrupt / logout or EOF / clear screen

7. Typical workflow for the solver

```
# local PC
scp -r .\pdptw_project fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/
ssh fhy0180@login23-1.hpc.itc.rwth-aachen.de
# on CLAIX
cd ~/pdptw_project && mkdir -p logs
# edit instances.txt (one path per line) and '#SBATCH --array=0-(N-1)' in run_extended_v4.sh
sbatch run_extended_v4.sh
squeue -u $USER
# when done, back on the local PC
scp -r fhy0180@login23-1.hpc.itc.rwth-aachen.de:~/pdptw_project/results ./results_claix
```

Docs: CLAIX – help.itc.rwth-aachen.de/en/service/rhr4fjjutttf/ | sciebo – help.itc.rwth-aachen.de/en/service/t84lrej1rt4i/